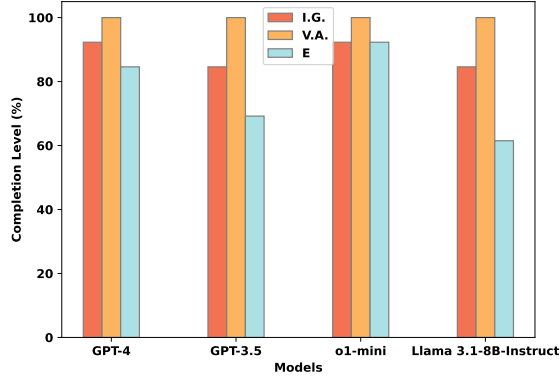
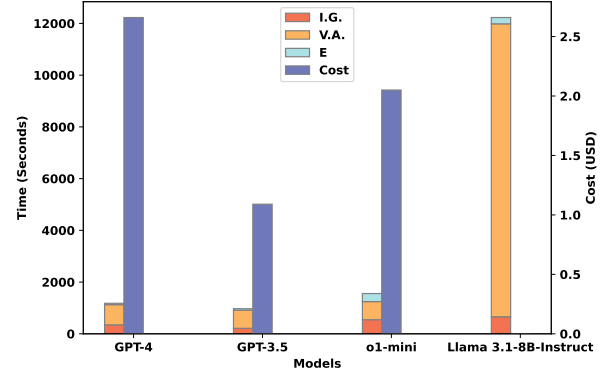




(a) Completion level on targets



(b) Average time spent and cost on targets

Figure 9: Completion level and overhead of different LLM Backbones.

4.6 Comparison with PENTESTGPT

We conducted a comparison of the effectiveness and efficiency of PENTESTAGENT against PENTESTGPT. Unlike PENTESTAGENT, PENTESTGPT requires human participation for feedback and decision-making throughout the penetration testing process. Thus, we compare their performance using case studies. We randomly selected ten vulnerabilities from VulHub (five easy, three medium, and two hard) and included 11 HackTheBox challenges (nine easy, one medium, and one hard). Two evaluators with different skill levels conducted the tests: an undergraduate student with limited penetration testing experience evaluated PENTESTGPT on the VulHub targets, while a PhD student with more experience assessed the HackTheBox challenges. Both systems were configured to use the GPT-3.5 model to ensure a fair comparison.

Fig. 10 shows the completion level and overhead comparison between PENTESTAGENT and PENTESTGPT on HackTheBox targets. Our results indicate that PENTESTAGENT achieves higher exploitation success and overall efficiency. For instance, PENTESTAGENT completes intelligence gathering in 220 seconds compared to 1199 seconds for PENTESTGPT, and finishes exploitation in 172 seconds versus 364 seconds. Although PENTESTGPT is slightly faster in vulnerability analysis, its slower performance in other stages due to human involvement reduces its overall efficiency. Additional results on VulHub targets and evaluation details can be found in the Appendix C. These findings imply that PENTESTAGENT can deliver more consistent and timely penetration testing, highlighting the benefits of minimizing human intervention in real-world security assessments.

4.7 Failure Analysis

We analyzed failure cases encountered during our evaluation and identified a few representative failure scenarios. As illustrated in Fig. 8, most failures occurred during the intelligence gathering and exploitation stages.

In the intelligence gathering stage, PENTESTAGENT occasionally fails to recognize services or applications with the appropriate level of granularity. For instance, our evaluation revealed that PENTESTAGENT struggled to detect components like PHPMailer, PHPUnit, and Ghostscript. These are not standalone applications but rather plugins or components running on web servers. Tools like Nmap

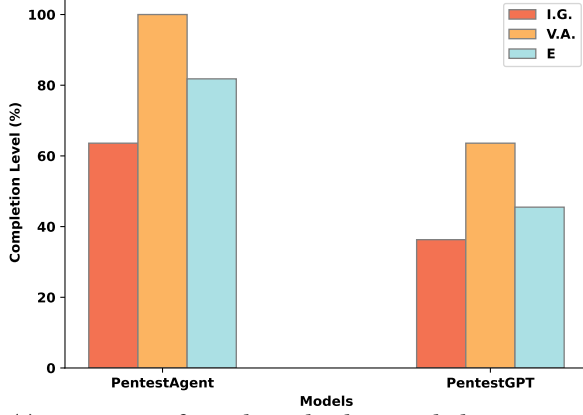
can identify the underlying web server frameworks, such as Nginx, but fail to enumerate these components. To address this limitation, PENTESTAGENT allows integration of additional web component fingerprinting tools and specialized libraries to more accurately detect and categorize such web components.

At the exploitation stage, PENTESTAGENT can encounter failures due to several challenges: requiring additional knowledge, needing user interaction, or experiencing LLM hallucinations.

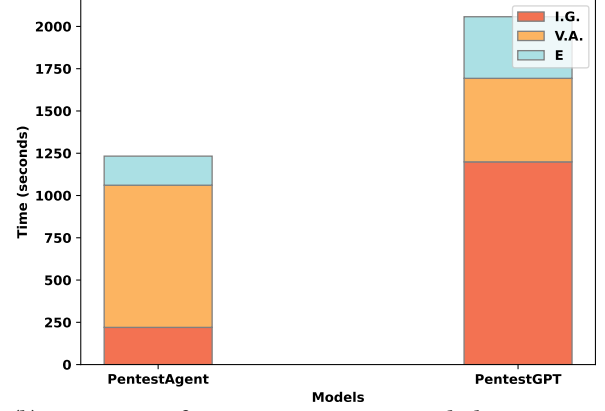
Requiring Additional Knowledge: Certain exploits demand a level of domain-specific knowledge that may exceed the capabilities of an LLM agent. For example, exploiting Samba server 4.6.3 (CVE-2017-7494) assumes the attacker has prior knowledge of credentials (username and password) to establish an SMB connection. Moreover, exploiting JBoss (CVE-2017-12149) requires expertise in using the "ysoserial" tool to craft payloads for exploiting unsafe Java object deserialization. These limitations can be overcome by integrating a human-in-the-loop design, where human experts can provide the additional knowledge or context required. Thanks to PENTESTAGENT's modular structure and its task-decomposition pipeline, human experts can easily intervene at any point in the testing process to assist with complex tasks.

Requiring User Interaction: Some exploits require user interactions that are typically performed manually, such as file uploads via web user interfaces. For instance, exploiting eFinder (CVE-2021-32682), an open-source file manager for web environments, involves manually creating and uploading an archive file. Similar to the mitigation method in the previous scenario, PENTESTAGENT allows the human user to step in at any penetration testing stage to assist tasks requiring user interaction. Furthermore, the recent advancements in intelligent agents like AutoGPT [4] offer a promising solution by mimicking human actions for complex tasks. By integrating such intelligent agents, PENTESTAGENT could automate these user interactions, significantly enhancing its capabilities in handling tasks traditionally performed by human testers.

LLM Hallucination: Another challenge is LLM hallucination, where the model generates incorrect or misleading information. This issue can be particularly problematic during the exploitation phase, as one hallucination can lead to a cascade of errors in subsequent steps. For example, if the execution agent fails to generate the correct commands or input parameters, it may mistakenly assume the exploit has bugs, leading it down an incorrect debugging path



(a) Comparison of completion levels on HackTheBox targets



(b) Comparison of average time spent on HackTheBox targets

Figure 10: Completion level and overhead comparison on HackTheBox targets.

that will never succeed. We employ several strategies to mitigate hallucinations. First, we reduce the randomness of LLM outputs by setting the model’s temperature to zero and attempting to execute the exploit multiple times. We also implement several stop conditions to prevent unintended consequences of hallucination, such as getting stuck in infinite loops or executing unintended actions. These stop conditions include hard-coded limits on the number of execution attempts and prompt-based conditions like “stop when you see the same error again.” Additionally, the attack knowledge base usually contains multiple exploits for the same vulnerability, allowing PENTESTAGENT to attempt different approaches until a functional exploit is found.

5 Discussion

5.1 Comparison with Existing Frameworks

While our primary comparison in this work is with PENTESTGPT, several emerging frameworks address related challenges in automated penetration testing. Notably, AutoAttacker and Enigma offer complementary perspectives that highlight different strengths and limitations relative to PENTESTAGENT.

AutoAttacker [48] focuses exclusively on the post-breach stage of an attack. It is designed to automate the “hands-on-keyboard” exploitation phase once a system has been compromised. In contrast, PENTESTAGENT addresses the entire penetration testing pipeline—from reconnaissance and vulnerability analysis to exploitation—providing a more comprehensive solution. This broader scope is critical for real-world scenarios, where early-stage tasks are just as vital as post-breach actions for assessing and improving security.

Enigma [2], on the other hand, extends the SWE-agent framework by integrating interactive tools that support solving Capture The Flag challenges. Its design primarily targets challenges in the crypto and reverse engineering domains and still relies on human-in-the-loop interactions. Although Enigma’s interactive interfaces are effective for guiding the agent through specific problem domains, its reliance on manual intervention limits full automation. In contrast, PENTESTAGENT is engineered to operate autonomously across diverse attack stages, reducing the need for human feedback and thus enabling more consistent performance in automated penetration testing.

Overall, these comparisons illustrate that while AutoAttacker and Enigma contribute valuable insights and capabilities, PENTESTAGENT distinguishes itself by offering an end-to-end automated solution.

5.2 Limitations on Performing Sophisticated Pentesting

Our system, PENTESTAGENT, focuses on exploiting individual vulnerable applications and services to help identify and mitigate these vulnerabilities. However, more sophisticated attack planning may be required in more complex penetration testing scenarios, such as red team simulations. These scenarios often involve combining several vulnerabilities to achieve a more challenging yet impactful exploit. For example, an SSRF vulnerability could be used as an intermediary step to exploit an internal application, eventually leading to obtaining root privileges.

While addressing such sophisticated attack strategies is beyond the scope of this paper, our framework, PENTESTAGENT, can still be valuable in these complex scenarios. Our system can identify and validate exposed vulnerabilities, such as SSRF, which can serve as starting points for further exploitation. This initial identification and validation process can significantly contribute to the overall penetration testing workflow, providing a foundation upon which more advanced exploitation techniques can be built.

6 Conclusion

This paper presents PENTESTAGENT, a novel LLM-based framework for automated penetration testing designed to address the limitations of existing frameworks: limited pentesting knowledge and insufficient automation. By leveraging a multi-agent architecture and incorporating various LLM techniques like retrieval augmented generation (RAG) and chain-of-thought (CoT), PENTESTAGENT enhances the penetration testing process through improved knowledge integration and automation.

Our comprehensive benchmark, based on VulHub’s vulnerable Docker environments and HackTheBox CTF challenges, provided a comprehensive test bed of PENTESTAGENT. The evaluation results demonstrate that PENTESTAGENT achieves strong performance in task completion and overall efficiency.

References

- [1] 0x727. 2024. ObserverWard. <https://github.com/0x727/ObserverWard>
- [2] Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E Jimenez, Farshad Khorrami, et al. 2024. EnIGMA: Enhanced Interactive Generative Model Agent for CTF Challenges. *arXiv preprint arXiv:2409.16165* (2024).
- [3] Paul Ammann, Duminda Wijesekera, and Saket Kaushik. 2002. Scalable, graph-based network vulnerability analysis. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*. 217–224.
- [4] AutoGPT. 2024. AutoGPT. <https://github.com/Significant-Gravitas/AutoGPT>
- [5] Mark S Boddy, Johnathan Gohde, Thomas Haigh, and Steven A Harp. 2005. Course of Action Generation for Cyber Security Using Classical Planning.. In *ICAPS*. 12–21.
- [6] Jinyin Chen, Shulong Hu, Haibin Zheng, Changyou Xing, and Guomin Zhang. 2023. GAIL-PT: An intelligent penetration testing framework with generative adversarial imitation learning. *Computers & Security* 126 (2023), 103055.
- [7] Rapid7 Global Consulting. 2019. Under the Hoodie: Lessons from a Season of Penetration Testing. <https://www.rapid7.com/research/reports/under-the-hoodie-2019/> Accessed: 2024-06-19.
- [8] Rapid7 Global Consulting. 2020. Under the Hoodie: Lessons from a Season of Penetration Testing. <https://www.rapid7.com/research/reports/under-the-hoodie-2020/> Accessed: 2024-06-27.
- [9] Alibaba Cloud. 2024. Vulnerability DB. <https://avd.aliyun.com/>
- [10] National Vulnerability Database. 2024. Common Vulnerability Scoring System Calculator. <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>
- [11] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Jailbreaker: Automated jailbreak across multiple large language model chatbots. *arXiv preprint arXiv:2307.08715* (2023).
- [12] Gelei Deng, Yi Liu, Victor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.
- [13] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [14] Matthew Denis, Carlos Zena, and Thaier Hayajneh. 2016. Penetration testing: Concepts, attack methods, and defense strategies. In *2016 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*. IEEE, 1–6.
- [15] Karel Durkota and Viliam Lisý. 2014. Computing Optimal Policies for Attack Graphs with Action Failures and Costs.. In *STAIRS*. 101–110.
- [16] GreenBone. 2024. GreenBone OpenVAS. <https://www.openvas.org/>
- [17] HackTheBox. 2024. Hackthebox: Hacking training for the best. <https://www.hackthebox.com/>
- [18] Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by ai: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2082–2086.
- [19] Zhenguo Hu, Razvan Beuran, and Yasuo Tan. 2020. Automated penetration testing using deep reinforcement learning. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2–10.
- [20] Leandri Krautsevich, Fabio Martinelli, and Artsiom Yautsiukhin. 2013. Towards modelling adaptive attacker's behaviour. In *Foundations and Practice of Security: 5th International Symposium, FPS 2012, Montreal, QC, Canada, October 25-26, 2012, Revised Selected Papers 5*. Springer, 357–364.
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [22] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems* 36 (2023), 51991–52008.
- [23] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2023. The Hitchhiker's Guide to Program Analysis: A Journey with Large Language Models. *arXiv preprint arXiv:2308.00245* (2023).
- [24] Michael Xieyang Liu, Frederick Liu, Alexander J Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J Cai. 2024. "We Need Structured Output": Towards User-centered Constraints on Large Language Model Output. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–9.
- [25] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhi Li, and Limin Sun. 2023. Harnessing the power of llm to support binary taint analysis. *arXiv preprint arXiv:2310.08275* (2023).
- [26] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*.
- [27] Microsoft. 2024. System message framework and template recommendations for Large Language Models (LLMs). <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/system-message>
- [28] MITRE. 2024. CVE. <https://cve.mitre.org/>
- [29] nmap. 2024. nmap. <https://nmap.org/>
- [30] Jorge Lucangeli Obes, Carlos Sarraute, and Gerardo Richarte. 2013. Attack planning in the real world. *arXiv preprint arXiv:1306.4044* (2013).
- [31] Forum of Incident Response and Inc. Security Teams. 2024. Common Vulnerability Scoring System v3.0: Specification Document. <https://www.first.org/cvss/specification-document>
- [32] Forum of Incident Response and Inc. Security Teams. 2024. Exploit Prediction Scoring System (EPSS). <https://www.first.org/epss/>
- [33] OWASP. 2024. OWASP Benchmark. <https://owasp.org/www-project-benchmark/>
- [34] OWASP. 2024. Top 10 Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- [35] Rapid7. 2024. Rapid7 Metasploit. <https://www.metasploit.com/>
- [36] Mark Roberts, Adele Howe, Indrajit Ray, Malgorzata Urbanska, Zinta S Byrne, and Janet M Weidert. 2011. Personalized vulnerability analysis through automated planning. In *Working Notes for the 2011 IJCAI Workshop on Intelligent Security (SecArt)*. 50.
- [37] Carlos Sarraute, Olivier Buffet, and Jörg Hoffmann. 2012. POMDPs make better hackers: Accounting for uncertainty in penetration testing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 26. 1816–1824.
- [38] Carlos Sarraute, Olivier Buffet, and Jörg Hoffmann. 2013. Penetration testing== POMDP solving? *arXiv preprint arXiv:1306.4714* (2013).
- [39] Carlos Sarraute, Gerardo Richarte, and Jorge Lucángeli Obes. 2011. An algorithm to find optimal attack paths in nondeterministic scenarios. In *Proceedings of the 4th ACM workshop on Security and artificial intelligence*. 71–80.
- [40] Snyk Security. 2024. Snyk Vulnerability Database. <https://security.snyk.io/>
- [41] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [42] The Penetration Testing Execution Standard. 2024. PTES Technical Guidelines. http://www.pentest-standard.org/index.php/PTES_Technical_Guidelines
- [43] Yaroslav Stefinko, Andrian Piskozub, and Roman Banakh. 2016. Manual and automated penetration testing. Benefits and drawbacks. Modern tendency. In *2016 13th international conference on modern problems of radio engineering, telecommunications and computer science (TCSET)*. IEEE, 488–491.
- [44] Tenable. 2024. Tenable Nessus. <https://www.tenable.com/products/nessus>
- [45] Vulhub. 2024. Vulhub. <https://github.com/vulhub/vulhub>
- [46] VulnHub. 2024. VulnHub. <https://www.vulnhub.com/>
- [47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [48] Jiachen Xu, Jack W Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. 2024. AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks. *arXiv preprint arXiv:2403.01038* (2024).
- [49] Tian-yang Zhou, Yi-chao Zang, Jun-hu Zhu, and Qing-xian Wang. 2019. NIG-AP: A new method for automated penetration testing. *Frontiers of Information Technology & Electronic Engineering* 20, 9 (2019), 1277–1288.

A Prompts

This section specifies more prompts used in the PENTESTAGENT pipeline.

The following prompt generates a structured output of the reconnaissance summary. Specifying the output structure and providing a comprehensive example guides the agent to output relevant information and reduces hallucination.

Reconnaissance Summary Prompt (Simplified)

Provide a summary of all reconnaissance findings ...
 The summary of findings should be presented in valid JSON format with the following fields: {FORMAT SPEC.}
 For example, {OUTPUT FORMAT EXAMPLE}

The following prompt summarizes the search results into a structured output for subsequent parsing and storing.

Search Results Summary Prompt

List ALL CVE numbers, URLs, keywords, and their applicable version relevant to exploit the vulnerabilities of {APP}. The results should be presented in valid JSON format with the following fields: {FORMAT SPEC.} ...
For example, {OUTPUT FORMAT EXAMPLE}

From our initial attempts, we found that the LLM is not familiar with software versioning. Therefore, we added a paragraph containing descriptions and examples to demonstrate how to handle software versions as few-shot learning. We use the following prompt to extract the desired information.

Exploit Procedure Analysis Prompt (Simplified)

RAG & CoT

Give a concise summary of the entire repository to answer the following questions:

- 1) whether this repository contains an exploit targeting a particular service or app;
- 2) What effect does the exploit have? Use one phrase to summarize the effect (e.g., remote command execution);
- 3) What relevant service/app version can this exploit be applied to?

Few-shot Learning

Note the app version is typically formatted as x.y.z. Explicitly state the version with the following formats ...

- 4) what are the requirements to run this exploit? (e.g., OS, library dependencies, etc.)

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} ...

For example, the response looks like this: {OUTPUT FORMAT EXAMPLE}

We designed the following prompt to generate a list of potential attack surfaces given a particular service or application.

Attack Surface Suggestion Prompt (Simplified)

List out all vulnerabilities ranked by confidence that can be used to exploit {app} {version} and provide the details about the vulnerabilities and the reasons to support each selection ...

The details should include ...

Make the selections by checking whether {version} is within the applicable version of the exploit and the vulnerability types ...

The results should be presented in valid JSON format with the following fields: {FORMAT SPEC.} ...

For example, {OUTPUT FORMAT EXAMPLE}

We designed the following prompt to generate a list of exploits for each potential attack surface.

Exploit Suggestion Prompt (Simplified)

List out paths of all relevant repositories ranked by the confidence that contain exploits ... applicable to {app} {version} and provide the details about the exploit and reasons to support each selection ...

The details should include ...

Make the selections by checking whether {version} is within the applicable version of the exploit and the execution effects ...

The results should be presented in valid JSON format with the following fields: {FORMAT SPEC.} ...

For example, {OUTPUT FORMAT EXAMPLE}

After obtaining a list of needed information, the execution agent uses the following prompt to query the environmental information database to fill in the information.

Execution Information Query Prompt (Simplified)

Based on the known information, try to provide the information listed here. {INFO NEEDED ...}

CoT

You should examine the information needed one by one. For each piece of information needed, you should ...

RAG

You should use your query tool to learn about the target environment ...

Structured Output

The results should be presented in valid JSON format with the following fields: {FORMAT SPEC.} ...

For example, the response looks like this: {OUTPUT FORMAT EXAMPLE}

B Benchmark Construction

B.1 VulHub Benchmark Construction

We use CVSS and EPSS scores to determine the difficulty of exploiting vulnerabilities. CVSS provides a numerical score reflecting the properties of vulnerabilities. Since most of the CVEs on VulHub adopt CVSS version 3.x metrics, we use this as our reference to assign difficulty levels. The numerical score is made of two parts: exploitability and impact. For our penetration testing purpose, we use the exploitability metric as the reference to assign difficulty levels. The exploitability score reflects the ease and technical means by which the vulnerability can be exploited [31]. A higher exploitability score indicates that the vulnerability is easier to exploit. We studied the distribution of exploitability scores, as shown in Fig.11. We found that most exploitability scores are above 3.0, and exploitability scores of 2.0 and 3.0 make natural cutoffs for easy, medium, and hard difficulties. Some vulnerable applications or services have more than one CVE number. We select the CVE to use based on the EPSS score. The EPSS scores measure how likely a vulnerability will be exploited in the wild. A higher EPSS score indicates the vulnerability is more likely to be exploited, making it more realistic

for penetration tasks. Fig. 12 shows the distribution of the EPSS scores of the CVEs in our benchmark dataset.

In addition, we remove the Docker images that are not associated with a CVE number and do not have CVSS 3.x scores. Additionally, some vulnerable applications are removed from the dataset due to complicated setup processes, such as requiring a license key from a service provider. To maintain integrity and fairness in our evaluations, we strictly prohibit PENTESTAGENT from directly accessing any content from VulHub repository, thereby preventing any advantage or bias in our testing methodology.

Fig. 14 shows the difficulty rating distribution of our VulHub benchmark dataset.

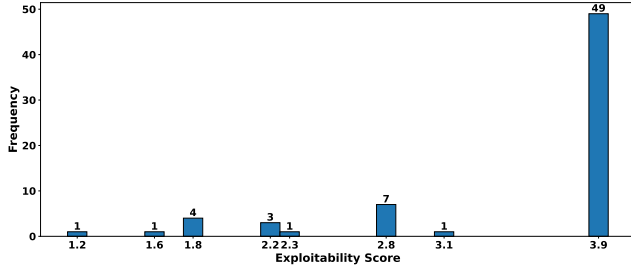


Figure 11: Distribution of exploitability scores

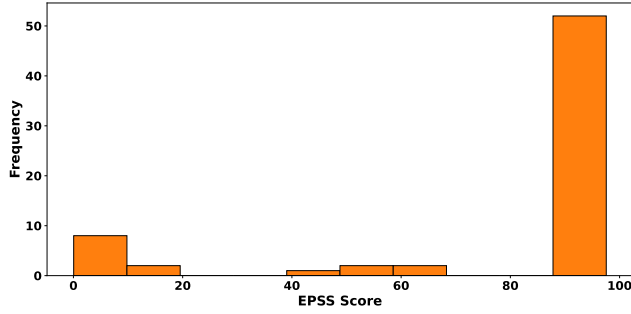


Figure 12: Coverage of EPSS scores

B.2 HackTheBox Benchmark Construction

In selecting HackTheBox (HTB) challenges for evaluating PENTESTAGENT, we aimed to create a diverse and representative set of targets that reflect real-world penetration testing scenarios. Our selection focused on key aspects such as operating system diversity, vulnerability relevance, and difficulty levels to ensure a comprehensive assessment.

To evaluate PENTESTAGENT's adaptability across different environments, we included both Linux and Windows machines. This diversity reflects the variety of systems encountered in real-world engagements and ensures that PENTESTAGENT is tested across different exploitation techniques. The chosen machines also incorporate well-known vulnerabilities spanning a decade, allowing us to assess PENTESTAGENT's ability to handle both historical and contemporary exploits. For instance, Blue features the EternalBlue

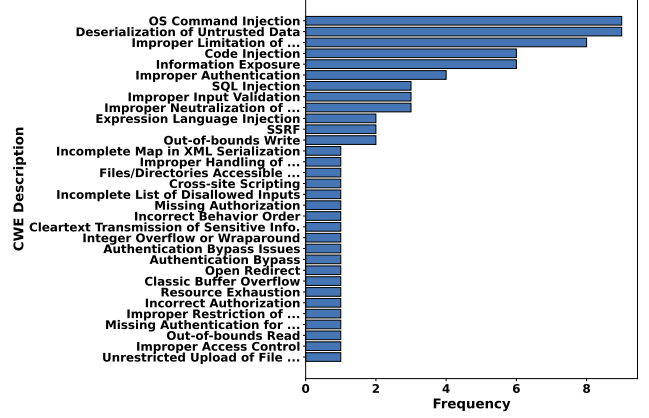


Figure 13: Coverage of CWE

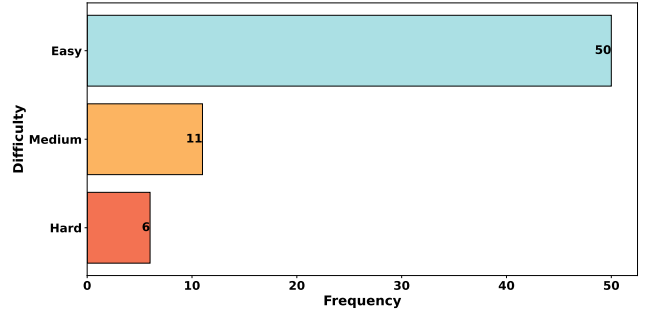


Figure 14: Distribution of exploitation difficulty ratings

vulnerability (CVE-2017-0144), a widely known Windows SMB exploit, while Legacy includes MS08-067 (CVE-2008-4250), another critical SMB-based attack.

We selected machines across easy, medium, and hard difficulty levels to analyze PENTESTAGENT's performance in different attack scenarios. Easier challenges, such as Lame and Optimum, test fundamental exploitation techniques, while medium and hard machines, such as Stratosphere (CVE-2017-5638) and Reel (CVE-2017-0199), require deeper reconnaissance, multi-step attacks, and more advanced reasoning. This progression ensures that PENTESTAGENT is evaluated not only on basic automation tasks but also on its effectiveness in handling complex, real-world pentesting challenges.

This carefully selected set of HTB challenges, as shown in Table 3, allows for a thorough assessment of PENTESTAGENT's automation capabilities, performance across different vulnerability types, and effectiveness in progressively complex penetration testing scenarios.

B.3 Benchmark Coverage

Our evaluation was conducted using a benchmark dataset comprising known vulnerabilities, which raises questions about the practicality in real-world scenarios. Firstly, it is important to recognize that known vulnerabilities pose significant risks. Many organizations and institutions struggle with timely patching practices,